Lab 9 – A First React App

Campus Coffee Cart

AI1220 — Software, Web & Mobile Engineering: Code without Limits

Goal

The coffee cart next to the library wants a small web page. It greets visitors, shows the menu with prices and counts the cups in an order. You build it in React from two components of your own and one state variable, and you watch the page change each time you save.

Set up

1. In a terminal, run `node -v`. You need version 22.12 or later. If the command is not found or the version is older, install the LTS version from https://nodejs.org and open a new terminal.
2. Extract `lab09_files.zip`. Rename the folder `lab09_files` to `lab09` and move it into your `ai1220` repository, so that `ai1220/lab09/src/App.jsx` exists. Move the whole folder, so the hidden `.gitignore` comes with it.
3. Open a terminal in `ai1220/lab09` (`cd` into it) and run `npm install` once (it needs the internet), then `npm run dev`. Open http://127.0.0.1:5175. Keep this terminal open: the page changes each time you save a file. Stop it with `Ctrl+C`.
4. In a second terminal in the same folder, run `npm test`. At the start all three test files fail and only four checks are listed; the other two appear once `Welcome.jsx` and `DrinkCard.jsx` exist. At the end, six checks pass.

You write `src/App.jsx` and two new files in `src/`: `Welcome.jsx` and `DrinkCard.jsx`. Use these names exactly, with the capital letters. Keep the other files as they are. AI assistance is allowed; check and understand your work.

Exercise 1: The welcome header — 20%

Create `src/Welcome.jsx` with a component `Welcome`, exported by default. It returns a `<header>` that holds, in this order:

- an `<h1>` with the text `Campus Coffee Cart`;
- a `<p>` with the text `Open every day from 8:00 to 16:00`

In `App.jsx`, import `Welcome` and show it as the first element inside `<main className="cart">`.

Exercise 2: The drink cards — 30%

Create `src/DrinkCard.jsx` with a component `DrinkCard`, exported by default. It receives two props: `name`, the name of the drink, and `price`, a number. It returns a `<section className="drink">` that holds an `<h2>` with the name, then a `<p>` with the price written as `AED 15` (for a price of 15). Everything the card shows comes from its props.

In `App.jsx`, after `Welcome`, add a `<div className="menu">` that holds three `DrinkCard` tags in this order: `Latte` at 15, `Tea` at 8 and `Hot chocolate` at 12.

Exercise 3: Count the cups — 30%

In `App`, add a state variable `cups` that starts at 0. After the menu, show a `<p className="order">` with the text `Cups ordered: 0`, where the number is the current value of `cups`. After that paragraph, add a `<div>` that holds a `<button>` with the text `Add a cup`. Each click adds 1 to `cups`, and the paragraph shows the new number at once.

Exercise 4: Clear the order, earn a cookie — 20%

In the same `<div>`, after `Add a cup`, add a second button with the text `Clear order`. A click puts `cups` back to 0. Between the order paragraph and the buttons, show a `<p className="offer">` only when `cups` is 3 or more, with the text `Free cookie with your order!`

Check your work

1. `npm test`: all six public checks pass.
2. In the browser, do these steps in order and compare.

Step	Expected
Open the page	The title and the opening hours; three cards: `Latte AED 15`, `Tea AED 8`, `Hot chocolate AED 12`; `Cups ordered: 0`; two buttons; no offer
Click `Add a cup` twice	`Cups ordered: 2`; no offer
Click `Add a cup` once more	`Cups ordered: 3`; `Free cookie with your order!` appears
Click `Clear order`	`Cups ordered: 0`; the offer is gone

Submission and marking

Run these in the top folder of your `ai1220` repository before the end of the lab:

```
git add lab09
git commit -m "Lab 9 coffee cart"
git push
```

Check on GitHub that `lab09/src` holds `App.jsx`, `Welcome.jsx` and `DrinkCard.jsx`, and that no `node_modules` folder was pushed. Each exercise earns its weight multiplied by the share of its checks that pass. The marking runs more checks of the same kind as `npm test`: for example, it draws `DrinkCard` with other drinks and prices, and it clicks the buttons more times. It also runs your `App` with correct `Welcome` and `DrinkCard` components, and the better result counts.